

SIMTiX: A Compact, Fully-Open SIMT Accelerator for RISC-V with an Energy-Aware Divergence Study

Krishna Gorai

Indian Institute of Technology Mandi

Email: kg05112001@gmail.com

Abstract—Open, readable hardware for studying Single-Instruction Multiple-Thread (SIMT) execution remains scarce: existing open GPGPU cores are large and difficult to bring up, while teaching cores rarely reach a real implementation flow. We present SIMTiX, a compact open-source SIMT accelerator attached to a five-stage RISC-V host through a memory-mapped offload interface. SIMTiX implements the full SIMT toolbox in a small, synthesizable design: eight lanes organized into hardware warps, a round-robin scheduler with a background memory engine for latency hiding, address coalescing on a cache-line-wide data port, a per-warp SIMT reconvergence stack for control divergence, and an on-chip scratchpad for data reuse. A key implementation result is that placing the vector register file and scratchpad in distributed RAM (LUTRAM) shrinks the design from 74% to 13% of the LUTs of a Xilinx Zynq Ultra-Scale+ device while closing timing at 100 MHz. We characterize SIMTiX with a six-kernel benchmark suite spanning streaming, stencil, reduction, and data-dependent control-flow workloads, verified bit-exact against a golden model at sizes up to 1024 threads. Against the scalar host CPU executing the same work, SIMTiX achieves 7–10 $\times$ speedup on streaming kernels, 2.8 $\times$ on a divergence-heavy kernel, and—reported honestly—a slowdown on a serial reduction, exposing where the SIMT model does and does not pay off. Using the SIMT active mask as a per-lane clock-enable, we quantify the lane-datapath dynamic energy a divergence-aware gated design saves (up to 22.5% on a divergent kernel). We further extend each lane with a pipelined floating-point unit supporting both single and half precision (FP32 and FP16): half precision reuses the single-precision datapath through NaN-boxed widen–compute–narrow, which we show is correctly single-rounded and adds no extra cycles. The unit is verified bit-exact against an IEEE-754 reference over 76,048 vectors, and sustains the engine’s full lane-parallel throughput. The complete RTL, benchmark harness, and FPGA flow are publicly available.

Index Terms—SIMT, GPGPU, RISC-V, hardware accelerator, control divergence, memory coalescing, floating-point, half precision, FPGA, clock gating, open-source hardware.

I. INTRODUCTION

Graphics processing units execute thousands of scalar threads in lockstep groups called *warps* under a Single-Instruction Multiple-Thread (SIMT) model. The microarchitectural mechanisms that make this efficient—warp scheduling for latency hiding, memory coalescing, control-divergence handling, and on-chip scratchpads—are well documented at the level of commercial GPUs, but they are hard to *study in RTL*. Production GPUs are proprietary; open GPGPU cores such as Vortex [1] are powerful but large and complex to bring

up; and the small academic cores are often simulator-only or stop short of a real implementation flow.

This paper presents **SIMTiX**, a deliberately compact open-source SIMT accelerator for RISC-V whose entire path—RTL, simulation, synthesis, place-and-route, bitstream, and post-route gate-level sign-off—is reproducible on a single workstation with free or widely available tools. SIMTiX is small enough to read end-to-end yet implements the complete SIMT mechanism set, and we use it to produce a measured, honest evaluation rather than a demonstration.

Contributions.

- A compact, fully-open SIMT accelerator (eight lanes, four resident warps) integrated with a five-stage RISC-V host as a self-contained system-on-chip, exercising the entire RTL-to-bitstream flow (Sections III–IV).
- An implementation study showing that moving the vector register file and scratchpad into distributed RAM reduces LUT usage by 5.7 $\times$ and closes 100 MHz timing, with a clear analysis of why the register-file read multiplexer, not the ALUs, dominates a naive SIMT datapath (Section IV).
- A six-kernel benchmark suite, verified bit-exact at up to 1024 threads, that isolates distinct microarchitectural behaviors, and a *measured* speedup against the scalar host—including a deliberately reported case where SIMT loses (Section VI).
- A divergence-aware lane clock-gating energy model that reuses the SIMT active mask as a per-lane clock-enable and quantifies the lane-datapath energy saved as a function of control divergence (Sections III, VI).
- A per-lane floating-point extension covering both FP32 and FP16, in which half precision is obtained almost for free from the single-precision path via NaN-boxed widen–compute–narrow (which we show is correctly single-rounded), verified bit-exact and shown to sustain the engine’s full lane-parallel throughput; we pipeline the unit and analyze its timing, finding that in the FP-enlarged fabric *interconnect*, not logic depth, bounds the achievable frequency (Section VII).

II. BACKGROUND: THE SIMT EXECUTION MODEL

A SIMT machine groups scalar *threads* into *warps* that share one instruction stream and execute on a vector of *lanes*. SIMTiX uses a warp size equal to its lane count

(`WARP_SIZE` = `NUM_LANES` = 8), so one warp occupies all lanes for one instruction. Multiple warps are resident simultaneously (`NUM_WARPS` = 4) and time-share a single fetch/decode/execute datapath; when one warp stalls on memory, the scheduler issues another, hiding latency. Three effects determine SIMT efficiency: *occupancy* (enough warps to hide latency), *memory coalescing* (merging lanes' accesses into wide transactions), and *control divergence* (lanes within a warp taking different branch outcomes, which serializes the divergent paths). SIMTiX implements hardware for all three and instruments each so they can be measured.

III. SIMTiX ARCHITECTURE

SIMTiX is an offload accelerator. The host RISC-V core programs a command block over a memory-mapped (MMIO) register interface—kernel entry PC, base pointers A/B/C, and thread count N—then writes a GO bit and polls a DONE status bit. On GO the accelerator launches a grid of N threads as $\lceil N/8 \rceil$ warps. Each thread receives its identity and arguments through a register-seeding convention at spawn (`a0`=thread id, `a1`..`a3`=base pointers, `a4`=N).

A. Warp pool, scheduler, and latency hiding

The warp pool holds four resident warp slots, each with its own architectural state (program counter, active mask, register file partition, and reconvergence stack), all sharing one datapath and one data port. A round-robin scheduler issues one runnable warp per cycle. Memory instructions are handed to a *background memory engine* that services the access while the scheduler keeps issuing *other* warps' compute instructions on the otherwise-idle datapath—the classic GPU latency-hiding trick. At most one memory instruction is in flight (single data port); a warp that wants memory while the port is busy simply waits its turn without blocking others.

B. Per-lane datapath and the register file

Each lane is a single-cycle RV32IM integer datapath (add/sub, logic, shifts, set-less-than, and mul; no divide or floating point). The vector register file (VRF) is logically $\text{NUM_WARPS} \times \text{NUM_LANES} \times 32$ words. A naive flip-flop VRF is the dominant cost of the design (Section IV); SIMTiX instead implements it as one distributed-RAM bank per lane, addressed by {warp, register}, with one synchronous write port and two asynchronous read ports.

C. Memory coalescing

The data port is one cache line wide (`LINE_WORDS` = 8, 256 bits). When a warp issues a load or store, the memory engine groups the active lanes' effective addresses by line and spends one transaction per distinct line. A contiguous warp access (e.g. `A[tid]`) collapses to a single line transaction, while a fully scattered access costs up to eight. A debug counter records line transactions, making coalescing efficiency directly measurable.

D. Control divergence: the reconvergence stack

Each warp owns a small SIMT reconvergence stack. The top-of-stack frame defines the warp's live PC and *active mask* (which lanes execute). A branch is resolved per lane: if all active lanes agree, the PC is simply retargeted; if they disagree, a frame is pushed so the two lane groups run in turn and reconverge at a join PC. SIMTiX supports the compiler-free convention of single-sided `if` and divergent loops: a forward branch reconverges at its target, a backward branch at the fall-through. A divergence counter records each divergent branch event.

E. On-chip scratchpad

A per-warp scratchpad (64 words/warp) occupies a dedicated address aperture. Accesses that fall in the aperture are serviced from on-chip RAM and never reach the global data port, so data reused across a warp's lanes (for example a broadcast matrix row) can be staged once and reused, cutting global traffic.

F. Divergence-aware lane clock-gating

The active mask is exactly the set of lanes doing useful work; masked-off lanes produce no architectural change yet, in a baseline design, still clock their pipeline registers. SIMTiX treats the active mask as a per-lane clock-enable: the mask drives an integrated clock-gate on each lane's registers, so only active lanes are clocked. Let an issued instruction activate a of L lanes. An ungated design clocks L lanes; a gated design clocks a . Summing over a kernel, the lane-datapath dynamic energy a gated design saves is

$$E_{\text{saved}} = 1 - \frac{\sum_i a_i}{L \cdot \sum_i 1} = 1 - \frac{\text{active-lane instructions}}{L \cdot \text{issued instructions}}. \quad (1)$$

Two hardware counters—issued instructions and summed active lanes—make Eq. (1) directly observable.

IV. IMPLEMENTATION

SIMTiX is written in synthesizable SystemVerilog and integrated with a reused five-stage RV32I host pipeline into a self-contained chip (`chip_top`): host core, driver ROM, accelerator, and an eight-bank on-chip shared memory. In the integrated chip the host loads the operands into shared memory, launches the accelerator, reads the results back, and publishes a checksum—the whole CPU→memory→accelerator→memory→CPU loop runs in hardware.

A. From flip-flops to LUTRAM

The first synthesis of the accelerator on a Zynq UltraScale+ (`xczu7ev`) consumed 170,868 LUTs—74% of the device—at only 10% of its flip-flops, and topped out near 122 MHz. The cause is structural: a flip-flop VRF of $4 \times 8 \times 32$ 32-bit words is 32,768 registers read as `vrf[warp][lane][reg]` with *both* warp and register variable, i.e. a 128-to-1 multiplexer per lane per read port. That multiplexer fabric, not the ALUs, dominates area and sets the critical path. Re-expressing the VRF as one distributed-RAM bank per lane, and likewise

TABLE I
ACCELERATOR PPA ON XILINX `xczu7ev` BEFORE/AFTER MOVING THE
VRF AND SCRATCHPAD TO DISTRIBUTED RAM (OUT-OF-CONTEXT
SYNTHESIS).

Metric	FF-based VRF	LUTRAM VRF+scratch	Δ
	baseline	after	
LUTs	170,868 (74%)	30,088 (13%)	−82%
Flip-flops	44,112 (10%)	4,590 (1.0%)	−90%
DSP48E2	24	24	—
BRAM	0	0	—
Timing (100 MHz)	WNS −3.21 ns	WNS +2.25 ns	met
Power	1.67 W @200 MHz	0.81 W @100 MHz	—

TABLE II
BENCHMARK KERNELS AND THE MECHANISM EACH ONE STRESSES.

Kernel	Computation	Stresses
vadd	$C_i = A_i + B_i$	baseline streaming
saxpy	$C_i = 3A_i + B_i$	coalescing, mul
fir	$C_i = A_i + 2A_{i+1} + A_{i+2}$	stencil, partial coalescing
relu	$C_i = \max(0, A_i)$	data-dependent light divergence
collatz	Collatz step count	data-dependent heavy divergence
reduce	$C_w = \sum_{l=0}^7 A_{8w+l}$	scratchpad + reduction

moving the scratchpad into LUTRAM, removes essentially all of those registers. Table I summarizes the effect: LUTs fall by 5.7× and the design closes 100 MHz timing with positive slack.

B. Full-chip place-and-route

The complete chip was taken through `opt/place/phys-opt/route` to a bitstream on `xczu7ev`. Post-route it meets all timing at 100 MHz (setup WNS +0.000 ns, hold met) and uses 34,850 LUTs (15.1%), 6,130 flip-flops (1.3%), 3,348 LUTRAM, 24 DSP, and zero BRAM, at 1.09 W. The accelerator is ∼87% of the logic; the host core and 4 KB shared memory are small by comparison.

C. Verification ladder

Correctness is established at four levels: (i) cycle-accurate RTL simulation with self-checking testbenches (Verilator); (ii) behavioral simulation in the vendor GUI; (iii) full place-and-route with post-route timing sign-off; and (iv) post-implementation *gate-level* functional simulation of the routed netlist (UNISIM primitives), which reproduces the integrated chip’s golden checksum of 964 bit-for-bit. The benchmark suite below adds a fifth, breadth-oriented level.

V. METHODOLOGY

A. Benchmark suite

We wrote six kernels, each chosen to stress a distinct mechanism (Table II). All are assembled from RISC-V source with a stock toolchain. Each runs at $N \in \{8, 64, 256, 1024\}$ threads (the divergence-heavy `collatz` to 256), and every output element is checked against a golden model computed in the testbench.

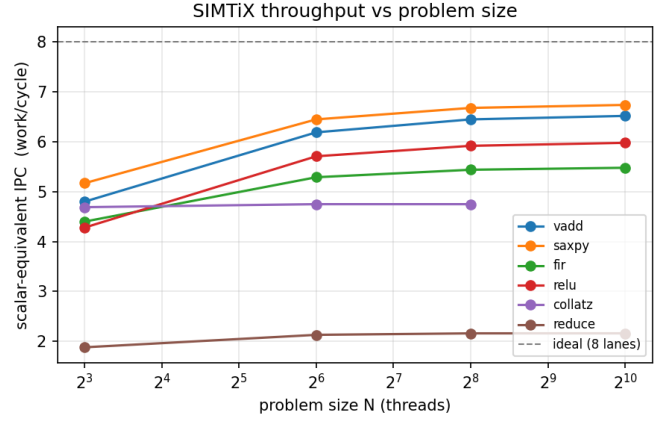


Fig. 1. Scalar-equivalent throughput versus problem size. Throughput ramps as occupancy rises (latency hiding) and saturates below the 8-lane ideal due to the single shared data port.

B. Metrics

Per run the engine reports cycles (launch to done), global line transactions, scratchpad transactions, divergent-branch events, issued instructions, and summed active lanes. From these we derive lane utilization $\sum a_i / (L \sum 1)$ and a scalar-equivalent throughput $\sum a_i / \text{cycles}$ (work retired per cycle).

C. Measured scalar baseline

For an apples-to-apples speedup, each kernel is also implemented as a natural scalar loop and executed on the *same* five-stage host pipeline in RTL, counting cycles from program start to completion. The host is pure RV32I, so constant multiplies are lowered to shift-and-add; `matmul`, which needs a general multiply, is therefore excluded from the scalar comparison. The scalar program and the accelerator are checked to produce identical results.

VI. RESULTS

A. Throughput and latency hiding

Figure 1 plots scalar-equivalent throughput against problem size. With only eight threads (one warp) throughput is low—there is nothing to hide memory latency behind—but it climbs as more warps become resident and saturates near 6.5–6.7 work-items/cycle for the coalesced kernels, out of an ideal of 8. The gap is the single data port: while one warp’s coalesced access occupies the memory engine, only three other warps are available to fill the datapath.

B. Memory coalescing

Figure 2 shows global line transactions versus size. `saxpy/vadd` are perfectly coalesced (three line transactions per warp—one each for A, B, C), whereas the `fir` stencil reads $A[i], A[i+1], A[i+2]$; the shifted reads straddle line boundaries and double its traffic. This is the expected signature of *partial* coalescing and explains `fir`’s lower speedup. A deliberately scattered access (stride 8) costs the full eight

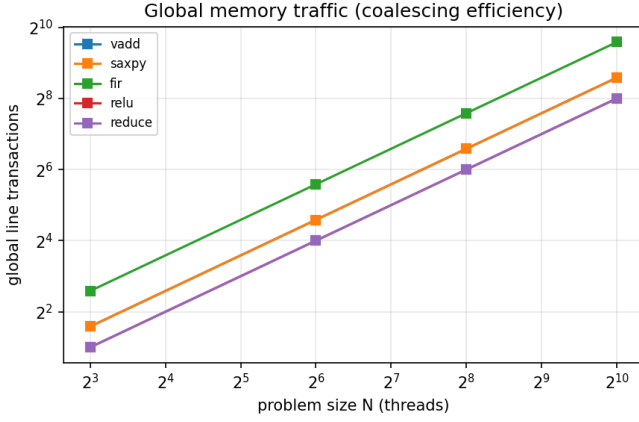


Fig. 2. Global line transactions versus size. Contiguous kernels coalesce to three transactions per warp; the *fir* stencil’s shifted reads straddle lines and roughly double the traffic.

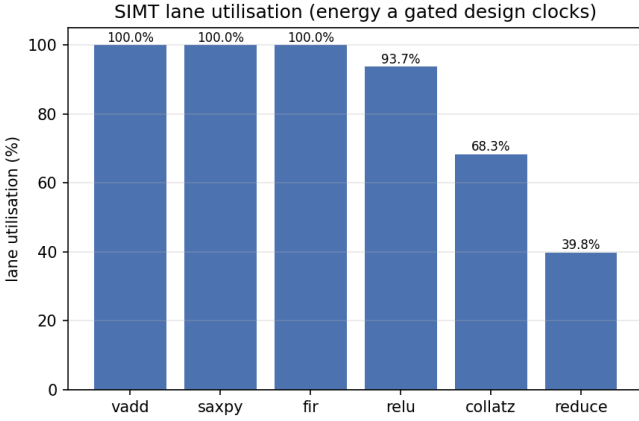


Fig. 3. SIMT lane utilisation per kernel (largest size). The complement is the lane-datapath dynamic energy a divergence-aware clock-gated design saves (Eq. (1)).

transactions per warp—an $8\times$ traffic increase that coalescing removes for contiguous data.

C. Control divergence and lane utilization

Figure 3 reports lane utilization. The convergent streaming kernels keep all eight lanes active every instruction (100%). *relu* diverges only where the input changes sign, dropping utilization slightly to 93.7%. *collatz*, whose per-lane trip count is data-dependent, falls to 68.3% as finished lanes mask off while others continue. *reduce* is lowest (39.8%) because its second phase collapses eight lanes’ partial sums on a single lane. Per Eq. (1), the unused fraction is the lane-datapath dynamic energy a clock-gated design saves: from 0% on convergent code up to 22.5% on a divergence-heavy microbenchmark.

D. Scratchpad data reuse

For an 8×8 matrix-row product, staging the broadcast input row in the scratchpad once and reusing it cuts global line

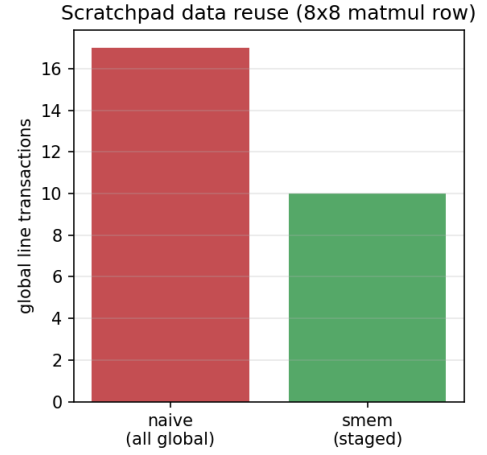


Fig. 4. Global line transactions for an 8×8 matmul row: staging the reused row in the scratchpad cuts off-chip traffic.

TABLE III
MEASURED CYCLES AND SPEEDUP (LARGEST SIZE; *COLLATZ* AT $N=256$). LANE UTILIZATION AND GLOBAL TRAFFIC FROM THE ACCELERATOR COUNTERS.

Kernel	N	Accel	CPU	Speedup	Lane-util
vadd	1024	1,413	13,322	$9.43\times$	100%
saxpy	1024	1,669	16,394	$9.82\times$	100%
fir	1024	2,053	14,345	$6.99\times$	100%
relu	1024	1,284	12,809	$9.98\times$	93.7%
collatz	256	9,637	26,601	$2.76\times$	68.3%
reduce	1024	6,023	4,363	$0.72\times$	39.8%

transactions from 17 to 10 (Figure 4). At this small size the staged version spends more cycles (the scratchpad is single-ported and serializes the stage-in), so the win is in off-chip traffic rather than latency—an honest trade that grows with data size and reuse distance.

E. Measured speedup over the scalar host

Figure 5 and Table III give the headline result: measured accelerator speedup over the scalar host executing identical work. Streaming kernels reach 7–10 $\times$, approaching the eight-lane width scaled by the coalescing traffic reduction. *fir* is lower (7.0 $\times$) because of its partial coalescing. *collatz* achieves 2.8 $\times$: heavy data-dependent divergence erodes—but does not eliminate—the SIMT advantage. *reduce* is below unity (0.72 $\times$): its single-lane collapse and single-ported scratchpad make it *slower* than the scalar core. We report this case deliberately—the SIMT model is not universally beneficial, and a reduction needs a better hardware primitive (e.g. a tree-reduction network) to win on this engine.

F. Design-space exploration: lanes and warps

SIMTiX is parameterized in its lane count and resident-warp count ($\text{WARP_SIZE} = \text{NUM_LANES}$, and the coalescing line scales with the lanes). We swept $\text{NUM_LANES} \in \{4, 8, 16, 32\}$ against $\text{NUM_WARPS} \in \{2, 4, 8\}$ —all twelve configurations elaborate and pass the suite bit-exact—and

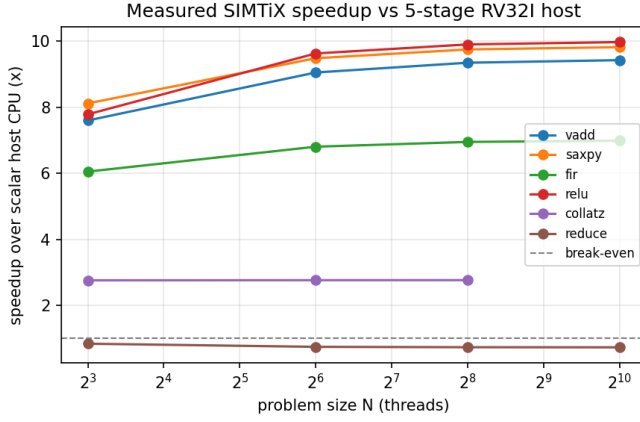


Fig. 5. Measured speedup over the five-stage RV32I host running the same work. Streaming kernels reach 7–10 $\times$; divergence reduces the gain; the serial reduction falls below break-even.

TABLE IV
GEOMETRIC-MEAN THROUGHPUT (WORK-ITEMS/CYCLE) OVER THE FIVE LANE-AGNOSTIC KERNELS, SWEEPED OVER LANES AND RESIDENT WARPS. ALL TWELVE CONFIGURATIONS VERIFY BIT-EXACT.

NUM_LANES	NUM_WARPS		
	2	4	8
4	3.06	3.06	3.05
8	5.85	5.85	5.85
16	11.68	11.68	11.65
32	23.24	23.24	23.14

measured throughput on the five lane-count-agnostic kernels. Table IV reports the geometric-mean throughput, and Figure 6 plots it against lane count.

Two clear effects emerge. First, throughput scales almost linearly with lanes: $3.06 \rightarrow 5.85 \rightarrow 11.68 \rightarrow 23.24$ work-items/cycle, about $1.97\times$ per doubling, i.e. $7.6\times$ from 4 to 32 lanes. Widening the lanes also widens the coalescing line, so global line transactions fall proportionally (e.g. *vadd* drops $768 \rightarrow 96$ transactions from 4 to 32 lanes). Second, and more interesting, the resident-warp count makes essentially no difference here (the three curves in Figure 6 coincide; eight warps is marginally *worse* due to extra spawn bookkeeping). This is honest about the harness: the behavioral memory model answers in a single cycle, so there is almost no latency for extra warps to hide; occupancy beyond the two warps needed to overlap the memory engine with the datapath pays off only under realistic multi-cycle memory. The two knobs are thus separable—lanes buy data-parallel throughput, warps buy latency tolerance—and both cost register-file area ($\propto \text{lanes} \times \text{warps}$); for the FP-enabled design we pair this with a placed area/frequency study in Section VII-H.

VII. FLOATING-POINT EXTENSION

The base engine is integer-only. We extend it with floating point in both single (FP32) and half (FP16) precision, keeping the same SIMT structure: the extension is a per-lane execute

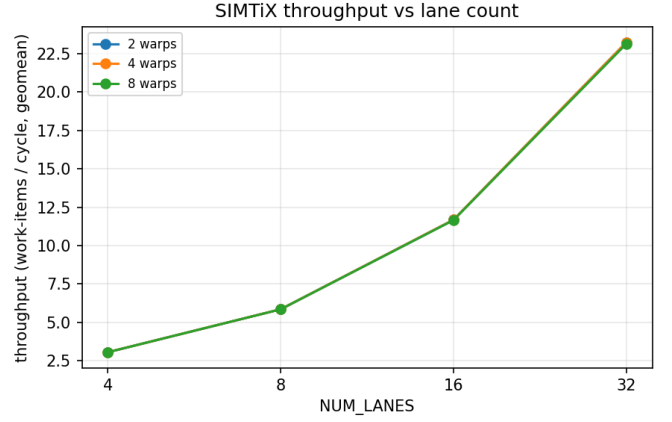


Fig. 6. Throughput versus lane count for each resident-warp count. Throughput scales near-linearly with lanes; the warp-count curves coincide because the single-cycle memory model offers little latency to hide.

unit and a second per-lane register file, not a new pipeline. It covers the full common arithmetic set—add, subtract, multiply, the fused multiply-add, and division and square root—plus the sign, compare, convert, and move operations, all in both precisions.

A. Per-lane FP unit and register file

Each lane gains a compact, fixed-latency floating-point unit beside its integer ALU, and a second per-lane register file holds the 32 floating-point registers in distributed RAM, addressed by {warp, register} exactly like the integer VRF. The two files have independent write ports, so an FP compute result and an integer load can retire in the same cycle; only same-file collisions defer. The unit implements add, subtract, multiply, the fused multiply-add (all four sign variants), sign-injection, min/max, the ordered compares, integer $\leftrightarrow$ float conversion, the FP32 $\leftrightarrow$ FP16 format conversions, the integer/FP bit-moves, and classify, in both precisions. Rounding is round-to-nearest-even; float-to-integer truncates; subnormals are flushed to zero in both formats, a deliberate throughput-oriented choice that keeps each lane small. Division and square root are inherently multi-cycle and are handled by a separate iterative unit behind a stall scoreboard (Section VII-D).

B. Half precision via NaN-boxed widen–compute–narrow

Half precision is not a second arithmetic datapath. A 16-bit half lives in a 32-bit register NaN-boxed (the upper 16 bits all ones); a mis-boxed value reads as a canonical NaN, per the RISC-V convention. For an FP16 operation the unit checks the box, widens the operand exactly to FP32, runs the single-precision adder/multiplier, and rounds the FP32 result back to FP16. This is correct, not approximate: because the FP32 intermediate carries a 24-bit significand and $24 \geq 2 \cdot 11 + 2$, rounding the exact result first to FP32 and then to FP16 gives the same value as rounding directly to FP16 [10], so FP16 add/sub/mul and integer $\leftrightarrow$ FP16 conversion are correctly

single-rounded. Sign-injection, min/max, comparison, classification, and the bit-moves are done natively at 16 bits so NaN payloads and the signalling/quiet distinction survive. Half precision thus reuses the verified single-precision logic almost entirely, adding only the widen, narrow, and box steps.

C. Fused multiply-add

The fused multiply-add computes $\pm(a \cdot b) \pm c$ with a *single* rounding, so it cannot be built by chaining the multiplier and adder (that would round twice). The unit forms the exact 48-bit product, anchors it in a wide fixed-point accumulator, aligns the addend to it while capturing a sticky bit, and normalizes and rounds once. When the addend out-ranges the product beyond the rounding window the product is provably below half a unit in the last place, so the result is the addend exactly—which also bounds the alignment shifter. It has fixed latency—unlike the iterative divide and square root—and commits through the ordinary writeback path; Section VII-H describes how this combinational chain is pipelined for timing closure. Half-precision FMA again uses widen–compute–narrow: the FP32 fused result of the widened operands, rounded once to FP16, is correctly rounded because the same $24 \geq 2 \cdot 11 + 2$ bound applies [10].

D. Multi-cycle divide and square root

Division and square root are produced one quotient/root bit per cycle by a digit-recurrence, so they are the engine’s first multi-cycle operations. The iterative core does restoring division of the significands and a restoring integer square root of the radicand scaled for the fractional bits. Because such a core is comparatively large, the engine has a *single* shared one rather than one per lane: when a warp issues a divide or square root the engine captures all of its lanes’ operands and parks the warp on a *stall scoreboard*, then sequences the active lanes through the shared core one at a time—the same park-and-resume mechanism the background memory engine uses for latency hiding, so the round-robin scheduler keeps issuing *other* warps onto the idle datapath meanwhile. The warp resumes once every active lane has been processed and its results write the f-file (the memory engine keeps write-port priority, so a result waits a cycle on a collision). At most one warp’s divide/square root is in flight, exactly as at most one memory access is; sharing the core trades latency— k active lanes take about k times as long, acceptable for these rare, already-hidden operations—for the area and congestion saving quantified in Section VII-H. Half precision reuses the FP32 core by the same widen–compute–narrow argument (the $24 \geq 2 \cdot 11 + 2$ bound holds for divide and square root too).

E. Verification

Each unit is verified standalone against a DPI-C IEEE-754 reference: FP32 against the host’s correctly-rounded `float` operations (including `fmaf` for the fused FMA and `sqrtof`), and FP16 against a `double`-based reference that is independent of, and was cross-checked against, the host `_Float16` type. Over random vectors spanning the product-dominant,

TABLE V
FLOATING-POINT OPERATION COVERAGE. EVERY OPERATION IS IMPLEMENTED AND CHECKED IN BOTH FP32 AND FP16; ALL CHECKS ARE BIT-EXACT AGAINST AN IEEE-754 REFERENCE.

Category	Operations (each in FP32 and FP16)
Arithmetic	<code>fadd</code> , <code>fsub</code> , <code>fmul</code>
Fused	<code>fmadd</code> , <code>fmsub</code> , <code>fnmsub</code> , <code>fnmadd</code>
Divide / root	<code>fdiv</code> , <code>fsqrt</code> (iterative, multi-cycle)
Sign / select	<code>fsgnj{,n,x}</code> , <code>fmin</code> , <code>fmax</code>
Compare	<code>feq</code> , <code>flt</code> , <code>fle</code>
Convert	<code>float↔int</code> (signed/unsigned), <code>FP32↔FP16</code>
Move / classify	<code>fmv.x._, fmv._.x</code> , <code>fclass</code>
Bit-exact checks vs IEEE reference: 116,089 (0 mismatches)	

balanced (cancellation), and addend-dominant regimes, plus directed infinity/NaN/zero/flush-to-zero/overflow, divide-by-zero, square-root-of-negative, mis-NaN-boxed-operand, and conversion corners, the pipelined unit passes 96,066 checks and the iterative divide/square-root unit a further 20,023, all bit-exact with zero mismatches (Table V). End-to-end kernels then run the complete warp pool: streaming add and multiply-add in each precision, a fused $C_i = A_i B_i + C_i$, and—through the stall scoreboard— $C_i = \sqrt{A_i/B_i}$ (two dependent multi-cycle ops per thread), all bit-exact to 1024 threads.

F. Floating-point throughput

The FP unit is pipelined over two stages (Section VII-H): an FP instruction commits one cycle after issue, and the issuing warp parks a single slot until it does. On streaming kernels this added latency is fully hidden by SIMT warp interleaving. Figure 7 plots throughput against problem size for streaming add and multiply-add kernels in both precisions; the curves show the same latency-hiding ramp as the integer suite and saturate near 7 work-items per cycle at full (100%) lane utilization, with coalesced traffic of three line transactions per warp. The extra pipeline stage costs throughput only where a dependent FP result is consumed immediately—the tight reductions of Section VII-G—and leaves these streaming kernels unaffected at scale. Critically, the FP16 curves lie exactly on the FP32 curves, and the cycle counts are identical (Figure 8): half precision shares the same pipelined datapath, so it costs no extra time. Half precision therefore delivers reduced storage and bandwidth and ISA-level FP16 support at *no* throughput penalty; its area and timing cost is part of the PPA discussion below.

G. FP application kernels

To exercise the whole datapath we run four toolchain-assembled FP kernels through the engine, each verified bit-exact (Table VI): `fsaxpy` ($C = \alpha A + B$, one fused multiply-add per element, in FP32 and FP16); `fnorm` ($C = A/\sqrt{B}$, both multi-cycle SFU operations per element); and `fdot`, a per-warp dot product that stages each lane’s product in the scratchpad and reduces them on lane 0—the building block of a fully-connected layer. Two effects stand out (Figures 9, 10). First, the iterative SFU is visible: `fnorm` takes about $6\times$ the

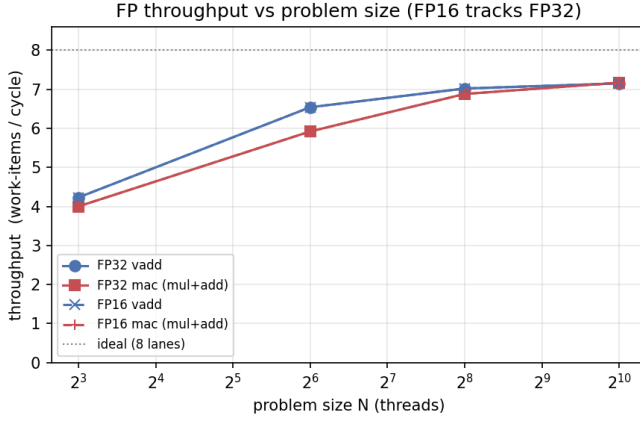


Fig. 7. Floating-point throughput versus problem size for streaming add and multiply-add kernels. The ramp is the same latency-hiding behavior as the integer suite; the FP16 curves coincide with the FP32 ones because both use the same pipelined datapath.

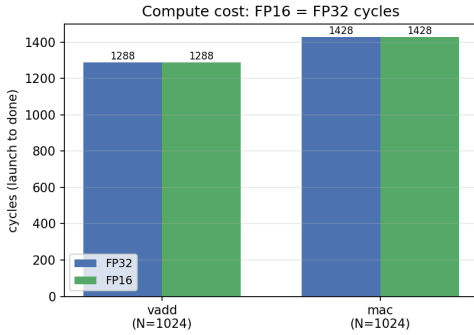


Fig. 8. Cycles to completion at $N=1024$: FP16 equals FP32 for both kernels, confirming half precision adds no compute-time cost on the shared FP datapath.

cycles of the streaming `fsaxpy` at the same size, the honest price of correctly-rounded divide and square root on a small iterative unit (well hidden across warps, but real). Second, divergence interacts with the now-larger lane datapath: `fdot`'s reduction leaves only one of eight lanes active during the collapse, so it runs at 45% lane utilization—and by Eq. (1) that idle 55% is lane-datapath dynamic energy a mask-gated design clocks away. Because the floating-point unit makes each lane substantially larger than the integer-only baseline, the *absolute* energy that divergence-aware gating saves on such kernels grows accordingly.

H. FP area cost

Table VIII gives the cost of the extension: an out-of-context synthesis of the FP-enabled accelerator on the same `xczu7ev`, against the integer-only baseline. Floating point raises LUTs from 30,088 to 74,342 (2.5 $\times$), flip-flops from 4,590 to 7,894, and DSP blocks from 24 to 40 (the FP multipliers), at 1.34 W. The increase is dominated by the common-case arithmetic unit—eight per-lane copies of the add/multiply/FMA/convert datapath, the fused multiply-add

TABLE VI
FP APPLICATION KERNELS (FP32 UNLESS NOTED), VERIFIED BIT-EXACT. CYCLES, GLOBAL LINE TRANSACTIONS, AND LANE UTILIZATION AT $N=256$.

Kernel	Computation	Cycles	Gmem	Lane-util
<code>fsaxpy</code>	$\alpha A_i + B_i$ (fmadd)	419	96	100%
<code>fsaxpy16</code>	$\alpha A_i + B_i$ (FP16)	451	96	100%
<code>fnorm</code>	$A_i/\sqrt{B_i}$ (div+sqrt)	2,396	96	100%
<code>fdot</code>	$\sum_l A_{8w+l} B_{8w+l}$	1,991	96	45%

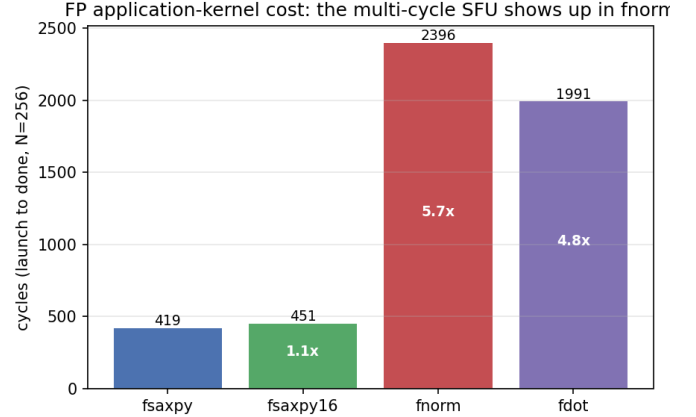


Fig. 9. Cycles at $N=256$ for the FP application kernels. The multi-cycle divide/square-root SFU makes `fnorm` about 6 $\times$ the streaming `fsaxpy`; `fdot` pays for its scratchpad reduction.

being the largest single block—rather than the iterative divide/square-root units, which synthesize to only 689 LUTs each ($\approx 5,500$ LUTs across eight lanes, $\sim 7\%$ of the design). Folding those eight onto one shared serial unit would therefore recover only a few percent of the area. The FMA is also the longest combinational chain in the design: as a single-cycle block its raw path is 16.2 ns (61 MHz, a conservative non-timing-driven estimate), making it the frequency bottleneck of the FP extension.

a) Timing closure. To recover frequency we pipeline the FP unit into two stages, split at the FMA accumulator (`mag`) boundary: the first stage forms the 48-bit product and aligns it into the 128-bit fixed-point accumulator, the second normalizes and rounds. The warp pool absorbs the extra cycle with a single-slot scoreboard (`W_FPC`)—the issuing warp parks one slot and resumes when the result commits, the same park-and-resume mechanism already used for memory accesses and the iterative divide/square-root unit, so no new structural hazard arises and SIMT interleaving hides the latency on streaming work (Figure 7). The pipeline registers raise flip-flops from 7.9k to 12.4k—the registered datapath is wide, eight lanes each holding a 128-bit accumulator—but leave LUTs, DSPs, and power essentially unchanged. Timing-driven synthesis with retiming cuts the FP *logic* depth from 16.2 ns to 5.3 ns; place-and-route on the `xczu7ev`, however, closes at 12.8 ns (**78 MHz**), short of the 100 MHz target. The placed critical path is still the first FMA stage, but it is now *routing*-

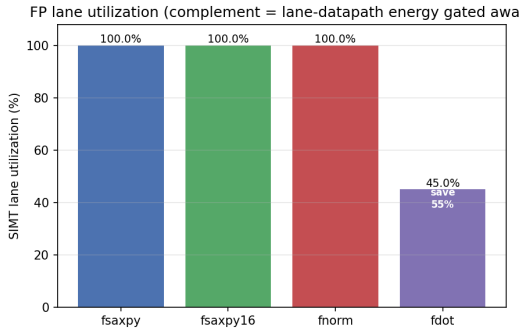


Fig. 10. SIMT lane utilization per FP kernel. Streaming kernels keep all lanes busy; `fdot`’s single-lane reduction idles 55% of the (now FP-enlarged) lane datapath, which mask-driven clock-gating removes.

dominated—7.4 of 12.8 ns (58%) is wire delay—because in the FP-enlarged fabric (2.5× the integer LUTs spread across eight wide FP lanes) interconnect, not logic depth, has become the binding constraint.

Acting on this diagnosis, we fold the eight per-lane divide/square-root cores into a single shared serial unit, fed lane-by-lane: one warp’s operation is in flight at a time and its active lanes are sequenced through the one core. This trades divide/square-root latency—rare, and already multi-cycle and parked—for area: it removes ~3.7k LUTs and ~1.2k flip-flops and drops power from 1.53 to 1.22 W. Crucially it leaves the FMA critical path structurally unchanged yet, by de-congesting the fabric, cuts that path’s *routing* delay from 7.4 to 6.5 ns, raising the placed maximum from 78 to **84 MHz** and reducing near-critical endpoints from 8.7k to 3.2k. Same logic, less congestion, higher frequency: a direct confirmation that interconnect, not logic depth, bounds this design.

b) Frequency does not scale with fabric size. If congestion is the limit, a natural question is whether *trimming* the FP fabric relieves it further. It does not. Placing the FP design at four lanes (Table VII) halves its area—37k versus 69k LUTs, 20 versus 40 DSPs—but reaches only 79.7 MHz, marginally *below* the eight-lane 84.1 MHz, and the router still reports CLB congestion. The reason is that the critical path is a *single* lane’s FMA cone, whose logic depth and locally-dense 128-bit datapath are independent of the lane count; halving the lanes replicates the same cone fewer times and packs the smaller design into a denser region that gives the lone FMA no relief. The eight-lane configuration therefore Pareto-dominates the four-lane one—equal frequency at twice the throughput—which both explains why the shared-SFU saving helped only modestly and identifies the true frequency lever as the FMA *microarchitecture* (a deeper FMA pipeline), not the fabric size. The area and power figures here are exact.

VIII. RELATED WORK

Open RISC-V GPGPU cores include Vortex [1], a full OpenCL-capable multi-core design, and Ventus [2], a PTX/Vortex-style GPGPU; both target performance and scale

TABLE VII
FP DESIGN-SPACE EXPLORATION: PLACED PPA OF THE FP-ENABLED ACCELERATOR ON `xczu7ev` VERSUS LANE COUNT (4 WARPS). AREA HALVES WITH THE LANES BUT THE PLACED FREQUENCY DOES NOT IMPROVE—THE PER-LANE FMA CONE IS THE FLOOR.

Lanes	LUT	DSP	Power	Placed Fmax	Thr. (w/cyc)
4	36,785	20	0.98 W	79.7 MHz	3.06
8	68,914	40	1.22 W	84.1 MHz	5.85

TABLE VIII
FP EXTENSION AREA COST: OUT-OF-CONTEXT SYNTHESIS OF THE ACCELERATOR ON `xczu7ev`, INTEGER BASELINE VERSUS FP-ENABLED.

Resource	Integer	FP-enabled	Δ
LUTs	30,088	74,342	2.5×
LUTRAM	1,428	3,348	—
Flip-flops	4,590	7,894	1.7×
DSP48E2	24	40	+16
BRAM	0	0	—
Power	0.81 W	1.34 W	—

and are correspondingly large. Simty [3] demonstrates generalized SIMT execution of the RISC-V ISA, and Nyuzi [4] is a synthesizable vector GPGPU for graphics research. Relative to these, SIMTix is deliberately small and readable, and its contribution is not raw performance but a complete, reproducible RTL-to-bitstream flow paired with a measured, mechanism-isolating evaluation and an energy-aware divergence study. Control-divergence handling via reconvergence stacks and dynamic warp formation is long studied [5], [6]; we use the standard stack mechanism and instead *measure* its energy implication through mask-driven lane clock-gating.

IX. DISCUSSION AND FUTURE WORK

The evaluation points to concrete next steps. The single shared data port caps throughput below the lane-count ideal; multiple memory ports or banked access would raise the saturation point. The *reduce* result motivates a dedicated cross-lane reduction primitive. The lane clock-gating model is analytic from counters; a gate-level power simulation would confirm the realized savings. The floating-point unit (Section VII) now covers the full arithmetic set including the fused multiply-add and iterative divide/square root in both precisions, with the divide/square-root cores folded onto a single shared serial unit that recovered both area and frequency (84 MHz post-route, Section VII-H). The FP design-space exploration then localized the remaining barrier: trimming the FP fabric does *not* raise the frequency (four lanes places below eight), because the critical path is a single lane’s FMA cone. The path to 100 MHz is therefore the FMA *microarchitecture*—a deeper, more finely balanced FMA pipeline—rather than fewer lanes or further area cuts; that, and an ASIC-flow re-balancing where wire delay is less dominant, are the clear next steps. The throughput design-space sweep (Section VI-F) establishes how performance scales with lanes and warps, and the FP PPA sweep (Section VII-H) completes the area/frequency/throughput picture. Finally, an open-source

ASIC path through the SkyWater 130 nm PDK [7] using OpenLane [8] would turn the FPGA PPA numbers into silicon-relevant area, frequency, and power.

X. CONCLUSION

SIMTiX shows that the full SIMT mechanism set—warp scheduling and latency hiding, memory coalescing, a reconvergence stack, and a scratchpad—fits in a compact, fully-open RISC-V accelerator that runs end-to-end through a real implementation flow. A distributed-RAM register file is the key to making it small and fast on FPGA. Across a verified six-kernel suite, SIMTiX delivers 7–10 $\times$ measured speedup where the SIMT model fits, degrades gracefully under divergence, and—reported honestly—loses on a serial reduction, while its mask-driven clock-gating saves up to 22.5% of lane-datapath dynamic energy on divergent code. A per-lane floating-point unit adds FP32 and FP16 arithmetic, with half precision obtained almost for free from the single-precision path by NaN-boxed widen–compute–narrow and verified bit-exact against an IEEE reference, sustaining the engine’s full throughput at no extra cycle cost. The design and its evaluation infrastructure are public, so the results are reproducible and the platform is reusable for teaching and research.

REFERENCES

- [1] B. Tine, K. P. Yalamarthy, F. Elsabbagh, and H. Kim, “Vortex: Extending the RISC-V ISA for GPGPU and 3D-Graphics,” in *Proc. IEEE/ACM MICRO*, 2021.
- [2] F. Zhang *et al.*, “Ventus: a RISC-V based GPGPU,” open-source project, 2022.
- [3] S. Collange, “Simty: Generalized SIMT execution on RISC-V,” in *Workshop on Computer Architecture Research with RISC-V (CARRV)*, 2017.
- [4] J. Bush, P. Dexter, T. N. Miller, and A. Carpenter, “Nyami: a synthesizable GPU architectural model for general-purpose and graphics-specific workloads,” in *Proc. IEEE ISPASS*, 2015.
- [5] W. W. L. Fung, I. Sham, G. Yuan, and T. M. Aamodt, “Dynamic Warp Formation and Scheduling for Efficient GPU Control Flow,” in *Proc. IEEE/ACM MICRO*, 2007.
- [6] E. Lindholm, J. Nickolls, S. Oberman, and J. Montrym, “NVIDIA Tesla: A Unified Graphics and Computing Architecture,” *IEEE Micro*, vol. 28, no. 2, 2008.
- [7] SkyWater Technology and Google, “SkyWater Open Source PDK (sky130),” <https://github.com/google/skywater-pdk>.
- [8] A. Ghazy and M. Shalan, “OpenLane: The Open-Source Digital ASIC Implementation Flow,” in *Workshop on Open-Source EDA Technology (WOSET)*, 2020.
- [9] A. Waterman and K. Asanović, “The RISC-V Instruction Set Manual, Volume I: Unprivileged ISA,” RISC-V International.
- [10] S. A. Figueroa, “When is double rounding innocuous?,” *ACM SIGNUM Newsletter*, vol. 30, no. 3, pp. 21–26, 1995.